Distributed File Replicator (DFR)

Dossier de référence pour l'équipe de développement

Auteurs : Djeuda Tchapnga Paul Hervé, Garka Garka Jules Joël. Encadrant : M. Nguimbus Emmanuel.

Version 1.0. Ce document unifie le README, le cahier des charges, le cahier d'analyse et le cahier de conception, et corrige leurs écarts (voir §3.6). En cas de contradiction, **ce document fait foi**.

1. Présentation du projet

1.1 Contexte et problématique

La synchronisation manuelle de fichiers entre machines est source d'erreurs et n'est pas réactive. Les architectures centralisées créent un point de défaillance unique (SPoF) et demandent une maintenance constante. Les solutions existantes sont souvent trop lourdes pour des besoins simples.

Problématique : comment assurer une réplication automatique, sécurisée et cohérente de fichiers entre plusieurs nœuds, sans intervention humaine, avec peu de ressources et une architecture simple ?

1.2 Vision

DFR est un service léger et open source qui réplique automatiquement un répertoire entre plusieurs machines Debian. Chaque nœud est à la fois source et destination (multi-maître). Une interface web permet à des non-spécialistes de superviser et contrôler le système sans ligne de commande.

DFR **orchestre des outils éprouvés** (inotify, rsync, SSH, systemd) plutôt que de réinventer le transport de fichiers.

1.3 Objectifs

- **Général :** répliquer automatiquement, rapidement et de façon fiable des fichiers entre plusieurs nœuds Debian.
- **Spécifiques :** détecter les changements en temps réel ; les propager à tous les nœuds ; assurer une réplication bidirectionnelle cohérente ; sécuriser les transferts via SSH ; résoudre les conflits (Last Writer Wins) ; rester léger et simple à administrer.
- **Pédagogiques :** comprendre la réplication, la cohérence et la propagation ; maîtriser la synchronisation distribuée ; réaliser un système proche de rsync.

1.4 Périmètre

Dans le périmètre (v1)	Hors périmètre (v1)
Un répertoire racine répliqué par nœud	Réplication de blocs ou de systèmes de fichiers entiers
2 à 5 nœuds Debian 12 en maillage complet	Plus de 5 nœuds, réseaux WAN instables
Création, modification, suppression, renommage	Verrouillage distribué de fichiers
Stratégies push (temps réel) et pull (réconciliation)	Fusion de contenu (merge) de fichiers en conflit
Supervision et contrôle via interface web locale	Applications mobiles, multi-utilisateurs avec rôles avancés
Paquet `.deb`, service systemd	Autres distributions, Windows, macOS

Important : la réplication n'est pas une sauvegarde. Une suppression ou une corruption se propage. C'est pourquoi DFR conserve une corbeille et les versions perdantes en conflit (§4.5, §4.6).

1.5 Parties prenantes et acteurs

Acteur	Rôle
Administrateur système	Installe, configure, gère les clés SSH et surveille logs et état des nœuds
Utilisateur final	Travaille normalement dans le répertoire surveillé, la synchronisation est transparente
Système DFR	Acteur automatique : détecte, réplique, résout les conflits, journalise
Équipe de développement	Conçoit, développe, teste, livre
Encadrant	Valide les livrables

1.6 Glossaire

- **Nœud :** machine Debian qui exécute le démon DFR.
- **Racine :** répertoire répliqué sur chaque nœud (ex. `/srv/dfr/data`).
- **Pair :** autre nœud du maillage.

- **LWW :** Last Writer Wins, la version la plus récente gagne.
- **Tombstone :** enregistrement d'une suppression, pour éviter la résurrection d'un fichier.
- **Réconciliation :** comparaison périodique complète entre pairs (pull).

2. Exigences

2.1 Exigences fonctionnelles

Priorités MoSCoW : M = indispensable, S = important, C = souhaitable.

ID	Exigence	Prio
F1	Détecter création, modification, suppression, renommage via inotify	M
F2	Répliquer automatiquement vers tous les pairs	M
F3	Transfert différentiel (rsync)	M
F4	Mode multi-maître bidirectionnel	M
F5	Push temps réel et pull de réconciliation	M
F6	Résolution de conflits LWW avec conservation de la version perdante	M
F7	Journalisation de toutes les opérations	M
F8	Authentification par clés SSH, utilisateur dédié	M
F9	Vérification d'intégrité après réplication	M
F10	Redémarrage automatique (systemd)	M
F11	Interface web : état des nœuds, logs, pause/reprise, resynchronisation	M
F12	Installation par paquet `.deb`	S
F13	Alertes en cas d'échec de synchronisation	S
F14	Export des logs (CSV/JSON), filtres par date, nœud, type	S
F15	Configuration des répertoires et pairs depuis l'interface	C

2.2 Exigences non fonctionnelles (mesurables)

Domaine	Exigence	Seuil de validation
Latence	Détection d'un changement	< 2 s
Latence	Réplication d'un fichier < 1 Mo (LAN)	< 5 s de bout en bout
Ressources	CPU du démon au repos	< 2 % (valeur cible à confirmer par mesure)
Ressources	Mémoire du démon pour 10 000 fichiers	< 50 Mo (cible à confirmer)
Robustesse	Reprise après indisponibilité d'un nœud	Rattrapage automatique au plus tard à la réconciliation suivante
Robustesse	Aucun SPoF	Le maillage fonctionne avec un nœud en moins
Sécurité	Transferts chiffrés, droits minimaux, intégrité vérifiée	Tests de sécurité du §10
Administration	Installation en moins de 15 min par nœud	Test d'installation sur Debian vierge
Maintenabilité	Modules indépendants, couverture de tests unitaires du cœur	⩾ 70 % (cible)

3. Architecture

3.1 Principe

Architecture **multi-maître en maillage complet, sans nœud central**. Chaque nœud exécute : un démon `dfrd` (C), un client de contrôle `dfr-ctl`, une API REST locale (Flask) et une interface web.

3.2 Vue en couches

1. **Interface :** supervision, monitoring, administration (HTML/CSS/JS).
2. **API :** Flask, expose le démon en REST sur l'interface locale.
3. **Contrôle :** socket Unix local entre l'API et le démon.
4. **Cœur DFR :** événements, conflits, application des changements, journalisation.
5. **Système :** inotify, système de fichiers, rsync, SSH.

3.3 Modules du démon `dfrd`

Module	Rôle
`main`	Initialisation, orchestration, gestion des signaux (SIGHUP = rechargement)
`config`	Lecture et validation de `/etc/dfr/dfr.conf`
`watcher`	Surveillance inotify récursive, ajout dynamique des sous-répertoires
`event_queue`	File d'événements, regroupement (debounce) par chemin
`state`	Index local SQLite : chemin, taille, mtime, SHA-256, hash du dernier état synchronisé, origine
`sync_engine`	Interrogation des pairs, construction et exécution des commandes rsync
`conflict_manager`	Détection et résolution des conflits, tombstones
`peer_monitor`	Vérification périodique de disponibilité des pairs
`control`	Serveur du socket Unix (`/run/dfr/dfr.sock`)
`logger`	Journal structuré (JSON lignes) + journald

Le client `dfr-ctl` est aussi le point d'entrée des commandes distantes (`stat` , `hash` , `apply-delete` , `apply-rename`) exécutées via SSH sur les pairs.

3.4 Flux nominal

1. Un utilisateur crée ou modifie un fichier dans la racine.
2. Le noyau notifie `watcher` (inotify).
3. L'événement entre dans `event_queue` ; après la fenêtre de regroupement (≈ 1 s) un lot de chemins est constitué.
4. `sync_engine` interroge chaque pair (`dfr-ctl stat`) pour détecter un conflit.
5. Sans conflit : rsync sur SSH envoie les chemins ; avec conflit : `conflict_manager` tranche (§4.5).
6. Le pair applique, met à jour son index et vérifie le hash.
7. Tout est journalisé ; l'interface web reflète l'état.

3.5 Technologies

Techno	Usage	Justification
Debian 12	OS cible	Stabilité, paquets `.deb`
C (gcc, make)	Démon `dfrd`	Accès direct à inotify, faible empreinte
inotify	Détection	Natif noyau, temps réel
rsync + SSH	Transport	Différentiel, chiffré, authentification par clés
SQLite	Index d'état	Atomique, sans serveur
systemd	Service	Redémarrage et supervision
Python 3 + Flask	API et interface web	Légèreté
Git / GitHub	Versionnage	Collaboration
cmocka, bats, pytest	Tests	Voir §10

3.6 Décisions d'architecture (corrections des cahiers)

#	Décision	Écart corrigé
D1	Transport = **rsync sur SSH**. Le protocole TCP propriétaire est abandonné	Conception parlait de sockets TCP propriétaires sans chiffrement, contredisant charges et analyse
D2	Push = temps réel par événements ; pull = réconciliation périodique	README parlait de push/pull, les cahiers de multi-maître
D3	LWW sur horodatage, avec NTP/chrony obligatoire et copie du perdant conservée	LWW seul expose à des pertes de données si les horloges dérivent
D4	Les suppressions passent par des tombstones	Absent des cahiers
D5	Anti-boucle par hash d'index + liste d'exclusion temporaire	Absent des cahiers
D6	Socket **Unix** entre Flask et le démon	Conception disait « socket local » sans précision
D7	Index d'état en SQLite	Non défini

Si l'équipe préfère un démon en Python, la structure en modules reste valable. Cette décision doit être actée en phase 0.

4. Fonctionnement détaillé

4.1 Détection (inotify)

- Événements écoutés : `IN_CLOSE_WRITE`, `IN_CREATE` (répertoires), `IN_MOVED_FROM` / `IN_MOVED_TO` (appariés par cookie), `IN_DELETE`. `IN_MODIFY` est ignoré pour ne pas déclencher pendant une écriture en cours.
- inotify n'est pas récursif : un watch par répertoire, créé dynamiquement à l'apparition d'un sous-répertoire.
- Si `IN_Q_OVERFLOW` survient, ou si la limite `fs.inotify.max_user_watches` est atteinte, le démon journalise une erreur et lance une **réconciliation complète**.
- Exclusions par défaut : `.dfr/`, `*.swp`, `*.tmp`, `.~lock.*`, fichiers temporaires rsync.

4.2 File d'événements et regroupement

Les événements sont dédupliqués par chemin pendant une fenêtre de 1 s (configurable) afin de regrouper les rafales. Latence maximale visée : 2 s pour la détection. Un lot est envoyé par pair, en parallèle.

4.3 Réplication

Pour chaque pair, le démon exécute :

```
rsync -a --partial --partial-dir=.dfr/tmp --relative --files-from=- \
  --exclude=.dfr/ \
  -e "ssh -i CLE -o BatchMode=yes -o ControlMaster=auto -o ControlPersist=60"
\
  RACINE/ dfr@PAIR:RACINE/
```

`ControlMaster` garde la connexion SSH ouverte pour tenir le seuil de 5 s. Rsync écrit dans un fichier temporaire puis renomme de façon atomique : un lecteur ne voit jamais un fichier tronqué. Les permissions et propriétaires sont conservés (`-a`) ; les ACL et xattrs sont optionnels (`-A -X`).

4.4 Anti-boucle

Sans protection, A envoie à B, B détecte l'écriture et renvoie à A, indéfiniment. Deux mécanismes combinés :

1. Après application d'un changement distant, le pair enregistre le hash dans son index. Sur événement inotify, si le hash courant égale le hash indexé, l'événement est ignoré.
2. Les chemins en cours d'application sont placés dans une liste d'exclusion temporaire (≈ 3 s).

4.5 Conflits

- **Définition :** un fichier est en conflit si, depuis la dernière synchronisation (hash enregistré), il a changé sur au moins deux nœuds.
- **Résolution :** la version de mtime la plus récente gagne. En cas d'égalité, le nœud dont l'identifiant est le plus petit (ordre lexicographique) gagne, ce qui rend le résultat déterministe.
- **Conservation :** la version perdante est copiée dans `.dfr/conflicts/AAAAMMJJ-HHMMSS-NOEUD/chemin`. Rien n'est détruit silencieusement.
- **Journal :** une entrée détaille chemin, nœuds, horodatages, gagnant.
- **Modification vs suppression :** la modification gagne si son mtime est postérieur à la suppression ; sinon la suppression s'applique et le fichier part en corbeille.
- **Prérequis :** chrony/NTP actif sur tous les nœuds. `peer_monitor` mesure l'écart d'horloge et alerte au-delà de 2 s.

4.6 Suppressions et renommages

- **Suppression :** enregistrée comme tombstone (chemin, date, nœud) conservée 7 jours. Elle est appliquée chez les pairs par `dfr-ctl apply-delete`. Le fichier supprimé est déplacé dans `.dfr/trash/` (rétention configurable, 7 jours par défaut) au lieu d'être effacé.
- **Renommage :** les événements `MOVED_FROM` / `MOVED_TO` appariés par cookie donnent `dfr-ctl apply-rename ancien nouveau`, sans retransférer le contenu. Si l'appariement échoue (déplacement hors de la racine), le cas est traité comme suppression plus création.
- Les tombstones empêchent un nœud resté hors ligne de ressusciter un fichier supprimé à son retour.

4.7 Réconciliation périodique (pull) et reprise

Toutes les 5 minutes (configurable), et à chaque retour d'un pair, chaque nœud compare son index avec celui des pairs (`dfr-ctl stat` sur l'arborescence, limité aux métadonnées) et rattrape les écarts avec les mêmes règles que le push. Cela couvre : pair hors ligne, événements perdus, débordement inotify, démarrage à froid.

Si un pair est injoignable, les lots restent en file avec relance à délai croissant (5 s, 15 s, 60 s, plafond 5 min), sans bloquer les autres pairs.

4.8 Intégrité

Après chaque transfert, le démon compare le SHA-256 local à celui du pair (`dfr-ctl hash`). Au-delà d'une taille configurable (100 Mo par défaut), la vérification se limite à taille + mtime + contrôle de bloc interne de rsync. Un écart déclenche un nouveau transfert (3 essais), puis une alerte.

4.9 Ajout d'un nœud (bootstrap)

1. Installer le paquet, générer la clé, l'échanger avec les pairs.
2. Déclarer les pairs dans la configuration et démarrer le service.
3. Le nœud lance une réconciliation complète en mode « rattrapage » : il reçoit l'existant et n'envoie rien tant qu'il n'est pas à jour, ce qui évite d'écraser le réseau avec un répertoire vide.

4.10 Journalisation

Format JSON lignes dans `/var/log/dfr/dfr.log` (copie vers journald). Champs : `ts` , `level` , `node` , `event` , `path` , `peer` , `result` , `duration_ms` , `bytes` . Rotation par logrotate (30 jours par défaut). Les accès à l'interface web et aux commandes de contrôle sont aussi journalisés.

4.11 Configuration

Fichier `/etc/dfr/dfr.conf` , rechargé par SIGHUP :

```
[node]
id = node-a
root = /srv/dfr/data

[sync]
debounce_ms = 1000
reconcile_interval_s = 300
max_hash_size_mb = 100
trash_retention_days = 7
exclude = *.swp, *.tmp, .~lock.*

[peers]
node-b = dfr@192.168.1.12
node-c = dfr@192.168.1.13
```

```
[ssh]
key = /var/lib/dfr/.ssh/id_ed25519

[logging]
level = info
retention_days = 30

[web]
bind = 127.0.0.1:8080
```

5. Interfaces

5.1 Socket de contrôle

`/run/dfr/dfr.sock` (mode 0660, groupe `dfr`). Protocole : une requête JSON par ligne, une réponse JSON par ligne. Commandes : `status`, `peers`, `logs`, `conflicts`, `pause`, `resume`, `resync`, `reload`.

5.2 API REST (Flask)

Méthode	Route	Rôle
GET	`/api/status`	État du nœud local (en service, pause, file, dernier succès)
GET	`/api/nodes`	État des pairs, écart d'horloge, dernière synchro
GET	`/api/logs`	Logs filtrés (`from`, `to`, `node`, `type`, `level`), paginés
GET	`/api/logs/export`	Export CSV ou JSON
GET	`/api/conflicts`	Conflits récents et versions conservées
POST	`/api/control/{pause|resume|resync|reload}`	Contrôle du démon

Écoute sur `127.0.0.1` par défaut. Toute ouverture réseau passe par un proxy HTTPS et une authentification (session, mot de passe haché).

5.3 Interface web

Écrans : tableau de bord (état global, nœuds en ligne/hors ligne, débit, file d'attente), nœuds, journaux (filtres et export), conflits (consultation et restauration de la version perdante), contrôle. L'interface s'adresse à des non-spécialistes : libellés clairs en français, pas de jargon, états colorés avec texte (jamais la couleur seule).

6. Sécurité

- Utilisateur système dédié `dfr` sans shell interactif, droits limités à la racine, `/var/lib/dfr`, `/var/log/dfr`, `/run/dfr`.
- Clés SSH `ed25519`, une par nœud. Dans `authorized_keys` de chaque pair : `restrict` et `command="/usr/lib/dfr/ssh-gate"`. Le script `ssh-gate` n'autorise que rsync en mode serveur limité à la racine et les sous-commandes `dfr-ctl` listées.
- Validation stricte des chemins (rejet de `..`, de chemins absolus et des liens sortant de la racine).
- Politique des liens symboliques : répliqués comme liens, jamais suivis hors de la racine.
- Durcissement systemd : `NoNewPrivileges`, `ProtectSystem=strict`, `ReadWritePaths` limités.
- Pare-feu : seul le port SSH entre nœuds est ouvert ; l'interface web n'est pas exposée par défaut.
- Intégrité vérifiée après chaque réplication (§4.8). Journal des accès conservé.

7. Déploiement et packaging

Le paquet `dfr_X.Y.Z_amd64.deb` contient : binaire `/usr/sbin/dfrd`, `/usr/bin/dfr-ctl`, `/usr/lib/dfr/ssh-gate`, l'application web, la configuration par défaut, l'unité systemd et la règle logrotate.

Scripts : `postinst` (création de l'utilisateur, répertoires, génération de la clé, activation du service), `prerm` (arrêt), `postrm` (purge optionnelle).

Unité systemd :

```
[Unit]
Description=Distributed File Replicator
After=network-online.target ssh.service
Wants=network-online.target

[Service]
User=dfr
ExecStart=/usr/sbin/dfrd --config /etc/dfr/dfr.conf
Restart=on-failure
RestartSec=3
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/srv/dfr /var/lib/dfr /var/log/dfr /run/dfr

[Install]
WantedBy=multi-user.target
```

Procédure d'installation par nœud : installer le `.deb`, régler chrony, échanger les clés (`scripts/setup-ssh.sh`), éditer la configuration, démarrer le service, vérifier avec `dfr-ctl status`. Mise à jour : `apt install ./dfr_nouvelle.deb` sans perte d'état. Réglage conseillé : `fs.inotify.max_user_watches` ⩾ 524288.

8. Organisation du dépôt et conventions

```
dfr/
├── daemon/        src/*.c, include/*.h, tests/ (cmocka), Makefile
├── web/           app.py, templates/, static/, tests/ (pytest)
├── packaging/     debian/ (control, postinst, prerm, postrm, dfr.service)
├── scripts/       install.sh, setup-ssh.sh
├── tests/         integration/ (bats), env/ (scripts de création des VM)
├── docs/          cahiers, architecture, ce dossier
└── README.md  LICENSE  CHANGELOG.md
```

- **Git :** branches `main` (stable, taguée), `develop`, `feature/*`. Pull request obligatoire avec une relecture. Messages de commit de type `feat:`, `fix:`, `docs:`, `test:`.
- **Code :** C99, `-Wall -Wextra -Werror`, vérification mémoire avec valgrind ; Python formaté avec `black` et vérifié par `flake8` ; scripts shell vérifiés par `shellcheck`.
- **Intégration continue :** à chaque PR, compilation, tests unitaires, lint, construction du `.deb` et `lintian`.
- **Définition de « terminé » :** code relu, tests passants, documentation à jour, aucune régression en intégration.

9. Phases de réalisation

Durée prévue : 6 semaines. Les dates calendaires sont à fixer par l'équipe ; les phases sont séquentielles, avec un point de validation à chaque sortie.

Phase	Semaine	Intitulé
0	S1	Cadrage et environnement
1	S2	Noyau : détection et réplication unidirectionnelle
2	S3	Multi-maître
3	S4	Conflits et robustesse
4	S5	Interface web et packaging
5	S6	Validation, optimisation, clôture

Phase 0 : cadrage et environnement

- **Conditions d'entrée :** cahiers disponibles, équipe constituée, accès aux machines ou à la virtualisation.
- **Tâches :** acter D1 à D7 ; mettre à jour README et cahiers ; créer le dépôt et la CI ; monter 3 VM Debian 12 (`node-a/b/c`) ; installer SSH, rsync, chrony ; échanger les clés ; écrire le squelette (Makefile, modules vides, test fictif).
- **Conditions de sortie :** 3 VM joignables par SSH sans mot de passe ; horloges synchronisées ; CI verte sur le squelette ; décisions d'architecture validées par l'encadrant.
- **Livrables :** README corrigé, cahier de conception mis à jour, scripts d'environnement.

Phase 1 : noyau

- **Conditions d'entrée :** sortie de phase 0 atteinte.
- **Tâches :** `config` , `logger` , `watcher` récursif, `event_queue` avec regroupement, `state` (SQLite), `sync_engine` (rsync A vers B), `dfr-ctl stat/hash` , tests unitaires associés.
- **Conditions de sortie :** IT-01 à IT-05 passent (création, modification, suppression, renommages A vers B) ; latence mesurée et consignée.
- **Livrables :** démon fonctionnel mono-sens, journal structuré.

Phase 2 : multi-maître

- **Conditions d'entrée :** phase 1 validée ; modèle de données de l'index figé.

- **Tâches :** maillage de N pairs, envois parallèles, anti-boucle, tombstones, `peer_monitor`, réconciliation périodique, mode rattrapage au démarrage.
- **Conditions de sortie :** IT-06 à IT-09, IT-12 et IT-13 passent ; aucune réplication en boucle sur 30 min d'observation.
- **Livrables :** réplication bidirectionnelle à 3 nœuds.

Phase 3 : conflits et robustesse

- **Conditions d'entrée :** phase 2 validée.
- **Tâches :** `conflict_manager`, conservation des versions perdantes, corbeille, vérification SHA-256, relances à délai croissant, gestion du débordement inotify et du disque plein, `ssh-gate`, durcissement systemd, détection de dérive d'horloge.
- **Conditions de sortie :** IT-10, IT-11 et IT-14 à IT-21 passent ; aucune perte de données dans les scénarios de panne.
- **Livrables :** moteur de réplication complet et sécurisé.

Phase 4 : interface web et packaging

- **Conditions d'entrée :** socket de contrôle stable et commandes documentées.
- **Tâches :** API Flask, tableau de bord, journaux avec filtres et export, écran de conflits, contrôle pause/reprise/resynchronisation, authentification, paquet `.deb` avec scripts, guide d'installation.
- **Conditions de sortie :** IT-22 à IT-24 passent ; installation sur Debian vierge en moins de 15 min ; revue d'ergonomie par une personne non technique.
- **Livrables :** interface web, paquet `.deb`, guide d'installation.

Phase 5 : validation, optimisation, clôture

- **Conditions d'entrée :** toutes les fonctionnalités M terminées ; gel du code (feature freeze).
- **Tâches :** campagne complète de tests (§10), mesures de performance et de ressources, corrections, valgrind et analyse statique, documentation finale (exploitation, dépannage, architecture), démonstration, rétrospective.
- **Conditions de sortie :** 100 % des tests d'intégration M passent ; seuils du §2.2 atteints ou écarts justifiés ; tag `v1.0.0` ; livrables remis et validés.
- **Livrables :** version 1.0.0, rapport de tests, documentation complète, présentation.

10. Stratégie de tests

10.1 Niveaux

Niveau	Outil	Cible
Unitaire C	cmocka	`config`, `event_queue`, `state`, `conflict_manager`
Unitaire Python	pytest	Routes de l'API, filtres de logs
Intégration	bats + scripts shell sur 3 VM	Comportement réparti réel
Performance	scripts de charge, `time`, `pidstat`	Seuils du §2.2
Sécurité	tests manuels et scriptés	§6
Acceptation	Scénarios CU1 à CU6 avec l'encadrant	Cahier d'analyse

10.2 Environnement d'intégration

3 VM Debian 12 sur un même réseau local. Outils de perturbation : `tc netem` (latence, pertes), coupure d'interface (`ip link set down`), `kill -9` du démon, décalage d'horloge via `chronyc` ou arrêt de chrony, remplissage disque avec une petite partition dédiée. Chaque test commence par un état propre (racines vidées, index réinitialisés) et est automatisé.

10.3 Tests d'intégration

ID	Scénario	Résultat attendu
IT-01	Créer un fichier sur A	Présent sur B et C en moins de 5 s
IT-02	Modifier un fichier sur A	Contenu identique (SHA-256) partout
IT-03	Supprimer un fichier sur A	Supprimé partout, présent dans `.dfr/trash`
IT-04	Renommer un fichier sur A	Renommé partout, sans retransfert du contenu

ID	Scénario	Résultat attendu
IT-05	Renommer ou déplacer un répertoire	Arborescence identique partout
IT-06	Créer 1 000 petits fichiers en rafale	Tous répliqués, pas de perte, charge CPU mesurée
IT-07	Fichier de 1 Go, modifier 1 Mo	Volume transféré très inférieur à 1 Go (delta)
IT-08	Créer sur B	Visible sur A et C (bidirectionnel)
IT-09	Modifier le même fichier sur A et B à 1 s d'écart	Version la plus récente partout, perdante dans `conflicts/`, entrée de log
IT-10	Égalité d'horodatage	Gagnant déterministe, identique sur tous les nœuds
IT-11	Modification sur A, suppression sur B en parallèle	Règle du §4.5 appliquée, aucune perte silencieuse
IT-12	Observer 30 min après une rafale	Aucun événement en écho, compteur de synchronisations stable
IT-13	Couper B, modifier sur A, rétablir B	B rattrapé avant la réconciliation suivante
IT-14	`kill -9` du démon	systemd redémarre en moins de 10 s, état cohérent, aucune perte
IT-15	Décaler l'horloge de C de 5 min	Alerte émise, comportement documenté
IT-16	Corrompre un fichier sur un pair	Écart détecté, retransfert, alerte après 3 échecs
IT-17	Clé non autorisée, commande hors périmètre, chemin avec `..`	Accès refusé et journalisé
IT-18	Provoquer `IN_Q_OVERFLOW`	Réconciliation complète, aucun écart final
IT-19	Disque plein sur B	Erreur journalisée, relances, aucun fichier tronqué
IT-20	Écriture lente d'un gros fichier (30 s)	Réplication seulement à la fermeture du fichier

ID	Scénario	Résultat attendu
IT-21	Permissions, propriétaire, liens symboliques	Conservés, aucun lien ne sort de la racine
IT-22	Ajout d'un 4e nœud vide	Reçoit l'existant, n'écrase rien
IT-23	Interface web : panne d'un pair, filtres, pause/reprise	État affiché correct, actions effectives
IT-24	Installation, mise à jour et purge du `.deb`	Aucune erreur, état conservé à la mise à jour

10.4 Mesures de performance

À consigner dans le rapport de tests : latence de détection et de réplication (médiane et 95e centile) pour des fichiers de 1 Ko, 100 Ko, 1 Mo ; débit pour 1 000 fichiers ; CPU et mémoire au repos et en charge ; octets échangés pour une modification partielle d'un gros fichier.

10.5 Critères d'acceptation globale

Tous les tests marqués M passent ; seuils du §2.2 atteints ; aucune perte de données dans les scénarios de panne et de conflit ; installation reproductible ; documentation suffisante pour qu'un administrateur tiers déploie le système seul.

11. Risques et parades

Risque	Impact	Parade
Dérive d'horloge	LWW choisit la mauvaise version	chrony obligatoire, alerte à 2 s, versions perdantes conservées
Boucles de réplication	Saturation CPU et réseau	Anti-boucle par hash (§4.4), test IT-12
Planning serré (6 semaines)	Livraison incomplète	Priorités MoSCoW, F15 et F13 sacrifiables, phases séquentielles
Complexité du C	Fuites, plantages	valgrind, revue de code, démon redémarré par systemd
Limite inotify	Événements perdus	Réglage système, réconciliation de secours

Risque	Impact	Parade
Suppression ou corruption propagée	Perte de données	Corbeille, tombstones, versions conservées
Connaissances SSH/réseau inégales	Retards	Scripts d'environnement, documentation pas à pas
Matériel limité	Tests peu réalistes	VM sur une seule machine suffisent pour la validation fonctionnelle

12. Ressources et budget

Budget prévisionnel du cahier des charges : **2 400 000 FCFA**, soit matériel 1 825 000 (MacBook M3 Pro 1 500 000 ; MacBook M1 300 000 ; box internet 25 000), forfait internet 50 000, et ressources humaines 525 000 (développeur front junior 75 000 ; administrateur système junior 300 000 ; testeur 150 000). Les rôles à tenir sont : développeur du démon, développeur web, responsable infrastructure et packaging, responsable des tests. L'équipe actuelle de deux personnes les répartit ; la répartition nominative est à décider en phase 0.

13. Points à ne pas négliger

- **Réplication ≠ sauvegarde :** prévoir en complément une sauvegarde indépendante.
- **Scalabilité :** un maillage complet croît en N^2; limiter à 5 nœuds en v1.
- **Fichiers ouverts :** ne répliquer qu'à la fermeture (`IN_CLOSE_WRITE`).
- **Cas particuliers :** noms Unicode, espaces et caractères spéciaux, casse, très gros fichiers, fichiers verrouillés, fichiers spéciaux (ignorés et journalisés).
- **Espace disque :** corbeille, conflits et journaux consomment de l'espace ; prévoir des quotas et un nettoyage automatique.
- **Exploitation :** guide de dépannage (SSH refusé, pair hors ligne, conflit répété, disque plein), procédure de retrait d'un nœud, procédure de restauration depuis la corbeille.
- **Observabilité :** exposer au minimum la taille de la file, le dernier succès par pair et le nombre de conflits.
- **Gestion des versions :** versionnage sémantique, `CHANGELOG.md` , compatibilité entre versions voisines pendant une mise à jour progressive des nœuds.
- **Licence et conformité :** vérifier la compatibilité des dépendances avec la licence du dépôt.
- **Dépôt :** supprimer `TEST` et `DFR_2.TXT` , ranger les cahiers dans `docs/` , corriger l'année académique (2023-2024) et vérifier le tableau budgétaire des cahiers.

- **Diagrammes :** refaire les diagrammes de la conception (cas d'utilisation, composants, séquence, déploiement) dans un format texte versionnable (Mermaid ou PlantUML).
- **Évolutions futures :** configuration depuis l'interface (F15), chiffrement des journaux, plus de nœuds, gestion fine des ACL, alertes e-mail.

14. Décisions ouvertes

Sujet	Options	Échéance
Langage du démon	C (prévu) ou Python	Phase 0
Format de configuration	INI (prévu) ou TOML	Phase 0
Rétention corbeille et tombstones	7 jours (proposé)	Phase 3
Intervalle de réconciliation	5 min (proposé)	Phase 2
Seuil de hash complet	100 Mo (proposé)	Phase 3
Authentification de l'interface web	Mot de passe local (proposé)	Phase 4